CHAPTER 9

STRUCTURED QUERY LANGUAGE (SQL)

CHAPTER ESSENCE AT A GLANCE:
Structured Query Language (SQL) serves as the universal declarative relational database standard for schema definition, record manipulation, transactional control, and aggregated data analysis.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **SQL Command Classifications:** SQL organizes database operations into Data Definition Language (DDL) for structure manipulation, Data Manipulation Language (DML) for record handling, and Data Control Language (DCL) for privilege management.

• **Schema Definition and Integrity Constraints:** DDL commands CREATE, ALTER, and DROP establish and modify table schemas, enforcing domain and relational rules using PRIMARY KEY, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints.

• **Data Manipulation and Selective Filtering:** DML commands INSERT, UPDATE, DELETE, and SELECT manipulate table data, using the WHERE clause with relational operators, logical operators, and pattern matching wildcards.

• **Aggregate Processing and Group Summarization:** Aggregate functions COUNT, SUM, AVG, MIN, and MAX compute statistical summaries across tuple sets, grouped into partitioned categories using GROUP BY and filtered using HAVING.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{SQL Language Subsets} = \frac{\text{DDL}: \text{CREATE, ALTER, DROP} \quad \text{(Schema Structure Manipulation)}}{\text{DML}: \text{SELECT, INSERT, UPDATE, DELETE} \quad \text{(Record Manipulation)}}$$

Functional SQL Command Stratification

$$\text{SQL Query Execution Order} = \text{FROM} \rightarrow \text{WHERE} \rightarrow \text{GROUP BY} \rightarrow \text{HAVING} \rightarrow \text{SELECT} \rightarrow \text{ORDER BY}$$

Declarative Query Evaluation Pipeline

$$\text{Aggregate Summary Functions} = \text{COUNT}(*), \quad \text{SUM(Column)}, \quad \text{AVG(Column)}, \quad \text{MIN(Column)}, \quad \text{MAX(Column)}$$

Standard Column Aggregation Operators

$$\text{Filter Comparison} = \frac{\text{WHERE Clause}: \text{Filters Individual Rows Before Grouping}}{\text{HAVING Clause}: \text{Filters Aggregated Group Records After GROUP BY}}$$

Row Filtering vs Group Aggregation

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Never confuse WHERE with HAVING: WHERE filters individual table rows before any grouping occurs; HAVING filters summarized groups after the GROUP BY operation.

• **Alert:** DROP TABLE permanently deletes the entire relation structure and all its data; DELETE FROM removes rows from the table while preserving the table schema.

• **Alert:** String and date values in SQL statements must always be enclosed in single quotes (e.g. 'Commerce', '2026-03-31'); numeric literals are written without quotes.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Differentiate between DDL and DML commands in SQL. Give two examples of each. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Data Definition Language (DDL): Commands that define, modify, and drop database table structures and schemas (e.g. CREATE, ALTER, DROP). **[1 Mark]**

• Data Manipulation Language (DML): Commands that retrieve, insert, modify, and delete the data records stored inside tables (e.g. SELECT, INSERT, UPDATE, DELETE). **[1 Mark]**

2. Distinguish between DROP TABLE and DELETE FROM statements in SQL regarding schema and data persistence. [Score: 2 Marks]

Answer & Key Valuation Steps:

• DROP TABLE Statement: Permanently removes the entire table schema structure, its column definitions, indexes, and all constituent records from the database. **[1 Mark]**

• DELETE FROM Statement: Removes rows from the table according to a WHERE clause (or all rows if omitted), while the table schema and structure remain intact. **[1 Mark]**

3. Explain the difference between WHERE clause and HAVING clause with suitable SQL query examples. [Score: 3 Marks]

Answer & Key Valuation Steps:

• WHERE Clause: Evaluates filter conditions against individual table rows before any grouping or aggregate calculation takes place (e.g. SELECT * FROM Student WHERE Marks > 80;). **[1.5 Marks]**

• HAVING Clause: Evaluates filter conditions against aggregate summary groups formed by the GROUP BY clause (e.g. SELECT Course, AVG(Marks) FROM Student GROUP BY Course HAVING AVG(Marks) > 75;). **[1.5 Marks]**

4. Describe five SQL aggregate functions: COUNT(), SUM(), AVG(), MIN(), and MAX() with their syntax and purpose. [Score: 3 Marks]

Answer & Key Valuation Steps:

• COUNT(): Returns total number of rows or non-null column values (e.g. COUNT(*) returns total rows). **[0.6 Mark]**

• SUM(): Computes the mathematical sum of all numeric values in a specified column (e.g. SUM(Salary)). **[0.6 Mark]**

• AVG(): Calculates the arithmetic average of numeric values in a specified column (e.g. AVG(Marks)). **[0.6 Mark]**

• MIN(): Identifies and returns the smallest minimum value in a specified column (e.g. MIN(Price)). **[0.6 Mark]**

• MAX(): Identifies and returns the largest maximum value in a specified column (e.g. MAX(Salary)). **[0.6 Mark]**

5. Write SQL statements to: (a) Create an Employee table with EmpId as primary key, EmpName, Department, and Salary. (b) Insert a record into Employee. [Score: 3 Marks]

Answer & Key Valuation Steps:

• CREATE TABLE Statement: CREATE TABLE Employee (EmpId INT PRIMARY KEY, EmpName VARCHAR(30) NOT NULL, Department VARCHAR(20), Salary DECIMAL(10,2)); **[1.5 Marks]**

• INSERT INTO Statement: INSERT INTO Employee (EmpId, EmpName, Department, Salary) VALUES (101, 'Anand', 'Commerce', 55000.00); **[1.5 Marks]**

6. Given an Items table with columns ItemCode, ItemName, Price, and Quantity, write SQL queries to: (a) Display items with Price between 500 and 1500. (b) Display items whose name starts with letter 'S'. (c) Display all items ordered by Price descending. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Range Query: SELECT * FROM Items WHERE Price BETWEEN 500 AND 1500; **[1 Mark]**

• Pattern Matching Query: SELECT * FROM Items WHERE ItemName LIKE 'S%'; **[1 Mark]**

• Sorting Query: SELECT * FROM Items ORDER BY Price DESC; **[1 Mark]**

7. Given a Student table with columns RollNo, Name, Batch, and Marks, write SQL queries to: (a) Find average marks in each Batch. (b) Display Batches having more than 30 students. (c) Update Marks of student with RollNo 105 to 95. (d) Add a new column Grade to the table. **[Score: 4 Marks]**

Answer & Key Valuation Steps:

• Query (a) - Group Aggregation: SELECT Batch, AVG(Marks) FROM Student GROUP BY Batch; **[1 Mark]**

• Query (b) - Group Filtering: SELECT Batch, COUNT(*) FROM Student GROUP BY Batch HAVING COUNT(*) > 30; **[1 Mark]**

• Query (c) - Record Modification: UPDATE Student SET Marks = 95 WHERE RollNo = 105; **[1 Mark]**

• Query (d) - Schema Alteration: ALTER TABLE Student ADD Grade CHAR(2); **[1 Mark]**